GitHub Actions — Category 10: Migration & Strategic Decisions

Complete Interview Deep-Dive for SRE / Platform / DevOps Engineers

> **Prerequisites:** Categories 1–9 assumed. Real-world Jenkins and CI/CD operations experience assumed. **Format per topic:** What → Why → How → Key Concepts → Interview Answers → Examples → Gotchas ⚠ → Connections

---

Table of Contents

- 10.1 GitHub Actions vs Jenkins — Honest Comparison
- 10.2 Jenkins to GitHub Actions Migration Patterns
- 10.3 GitHub Actions vs GitLab CI, CircleCI, Tekton
- 10.4 Enterprise GitHub Actions Governance
- Cross-Topic Interview Questions
- Quick Reference Cheat Sheet

10.1 GitHub Actions vs Jenkins — Honest Comparison

What This Topic Is About

Interviewers at product/cloud companies almost always ask some version of: *"You have Jenkins experience — how does GitHub Actions compare, and when would you choose one over the other?"* This is not a trick question looking for "GHA is better." It requires genuine understanding of both tools' architectural tradeoffs.

The honest answer requires knowing: where Jenkins is still genuinely stronger, where GHA is genuinely stronger, and what factors in a real organization tip the decision one way or the other.

Architectural Foundations — The Core Difference

```
JENKINS: A long-running server application
   ┌──────────────────────────────────────────┐
   │  Jenkins Master (JVM process)            │
   │  - Stores all job configurations         │
   │  - Manages build queue                   │
   │  - Hosts web UI + REST API               │
   │  - Runs on your infrastructure 24/7      │
   │  - You patch, backup, upgrade it         │
   └──────────────────────────────────────────┘
                      │
                      │ agent protocol (JNLP/SSH)
          ┌───────────┴────────────┐
```

```
      Agent 1    Agent 2    Agent 3
    (your VMs/containers — you manage them)

  GITHUB ACTIONS: An event-driven orchestration service
     ┌──────────────────────────────────────────┐
     │  GitHub's orchestration service          │
     │  - GitHub manages this infrastructure    │
     │  - No server for you to run              │
     │  - Workflow config lives in the repo     │
     │  - Scales automatically                  │
     └──────────────────────────────────────────┘
                        │
                        │ long-poll HTTP (runner initiates)
            ┌───────────┼────────────┐
    GitHub-hosted  Self-hosted  ARC pods
    (ephemeral     (your infra, (K8s — you
     VMs, GH       you manage)  manage)
     manages)
```

Consequence of this difference: Every other comparison flows from here. Jenkins being a server means it has state, needs maintenance, and offers plugin extensibility. GHA being a service means no operational burden, but less extensibility and more GitHub coupling.

Feature-by-Feature Comparison

Compute and Runners

Dimension	Jenkins	GitHub Actions
Hosted compute	You provision everything	GitHub provides ephemeral VMs (ubuntu, windows, macOS)
Ephemeral agents	Requires Kubernetes plugin or Docker plugin configuration	Native — GitHub-hosted runners are ephemeral by default
Persistent agents	Default behavior (long-lived nodes)	Opt-in (self-hosted without --ephemeral)
Agent sizing	Any VM/hardware you configure	Fixed tiers (2-core to 64-core for GitHub-hosted); any for self-hosted
macOS	Requires your own macOS hardware or CI cloud	GitHub-hosted macOS (expensive: $0.08/min)
GPU/ARM	Your hardware	GitHub-hosted ARM64 + self-hosted GPU
Air-gapped	Works natively (no external calls required)	Requires self-hosted runners; controller still calls github.com
On-prem only	Native	Self-hosted runners only; GitHub.com required for control plane

Configuration and Code

Dimension	Jenkins	GitHub Actions
Config format	Jenkinsfile (Groovy DSL or Declarative)	YAML (`.github/workflows/*.yml`)
Config location	In-repo Jenkinsfile OR Jenkins UI (not in git)	Always in-repo — no UI-only configuration
Reuse mechanism	Shared Libraries (Groovy functions + pipeline templates)	Composite Actions (step reuse) + Reusable Workflows (job/pipeline reuse)
Scripting power	Full Groovy — loops, classes, arbitrary logic in pipeline	Limited expression language; complex logic requires shell steps
Dynamic pipeline	Full programmatic generation (loops, conditionals, stages)	Constrained — dynamic matrix for parallelism, but YAML is static
Multi-repo pipelines	Relatively easy with cross-job triggers	Possible via `repository_dispatch` / `workflow_call` but more friction

Security and Auth

Dimension	Jenkins	GitHub Actions
Secret storage	Jenkins Credentials store (per-job, per-folder, global)	GitHub Secrets (repo, org, environment scopes)
Cloud auth	Credentials plugins (stored long-lived keys)	OIDC keyless auth — no stored cloud credentials
Secret rotation	Manual + plugin-dependent	Manual; OIDC eliminates rotation for cloud credentials
Runner isolation	Workspace cleanup between builds (persistent agents)	Full VM isolation (GitHub-hosted ephemeral)
SCM trust	Jenkins controls what runs on what agent	GitHub controls via runner labels + runner groups
Audit	Jenkins Audit Trail plugin (limited)	GitHub org audit log (comprehensive, streamable)

Integration and Ecosystem

Dimension	Jenkins	GitHub Actions
Plugins	1,800+ plugins, mature ecosystem	Actions Marketplace (20,000+), younger but growing
GitHub integration	Requires GitHub plugin + webhook configuration	Native — deep integration, PR status checks built-in

Dimension	Jenkins	GitHub Actions
PR annotations	Requires plugins (Checkstyle, etc.)	Native `::error file=::` workflow commands
Notification	Email plugin, Slack plugin, etc.	Native + actions/github-script for rich integration
Multi-SCM	Native (Git, SVN, Mercurial, TFVC, etc.)	GitHub only (gitlab.com, bitbucket not supported)
Artifact storage	Build artifacts (local), external via plugins	GitHub Artifacts (90-day), GitHub Packages (GHCR)

Operations and Maintenance

Dimension	Jenkins	GitHub Actions
Operational burden	High — you run, patch, backup, upgrade Jenkins	Near-zero for hosted; Self-hosted runners you manage
HA/DR	You implement (active/active with external DB)	GitHub's SLA (99.9%) + GitHub status page
Plugin compatibility	Major version upgrades break plugins routinely	N/A — no plugin management
Backup	You backup Jenkins home, credentials, configs	Config lives in git; secrets not recoverable
Scaling	You scale agent fleet manually or via cloud plugins	GitHub-hosted auto-scales; ARC auto-scales on K8s
Visibility	Jenkins Blue Ocean UI, build history	GitHub Actions UI, PR status checks, step summaries

When Jenkins Is Genuinely Better

Be honest in interviews — this shows maturity:

```
1. COMPLEX DYNAMIC PIPELINE LOGIC
   Jenkins Scripted Pipeline is Turing-complete Groovy.
   You can write arbitrary loops, conditionals, classes, and
   programmatically generate any pipeline structure at runtime.

   GitHub Actions YAML is static (with constraints). Dynamic matrix
   helps, but genuinely complex logic (e.g., "query a database and
   generate a stage for each result row with its specific config")
   requires workarounds that are architecturally awkward in GHA.

   Example: A pipeline that reads from a service catalog API and
   generates 47 different deploy stages with per-stage configs
```

```
   is natural in Scripted Pipeline; it's a custom orchestrator
   in GitHub Actions.

2. NON-GITHUB SCM
   If your code is on GitLab, Bitbucket, Azure DevOps, or an
   on-premises git server, GitHub Actions is not available.
   Jenkins supports any SCM with a plugin.

3. AIR-GAPPED / FULLY ON-PREMISES
   Jenkins runs entirely in your network — no external calls needed.
   GitHub Actions requires github.com for the control plane.
   Self-hosted runners still phone home to GitHub APIs.
   Enterprises with classified or fully air-gapped requirements
   need Jenkins (or Tekton, or Concourse) — not GitHub Actions.

4. LEGACY PLUGIN ECOSYSTEM
   Some specialized integrations exist only as Jenkins plugins:
   - IBM zOS and mainframe build tools
   - Certain EDA/ASIC toolchains
   - Enterprise ALM systems (HP ALM, Micro Focus, etc.)
   If your workflow depends on a plugin that has no GitHub Action
   equivalent, you're either writing a custom Action or staying
   with Jenkins.

5. MIXED-SCM ENTERPRISE
   An org with GitHub, GitLab, Bitbucket, AND on-prem git —
   Jenkins as a single pane of glass across all SCMs is reasonable.
   GitHub Actions only works with GitHub.

6. DEEPLY CUSTOMIZED PIPELINE UI
   Jenkins Blue Ocean, custom views, and pipeline visualization
   are mature. Some enterprises have built significant internal
   tooling around Jenkins' API and UI. Migration cost is real.
```

When GitHub Actions Is Genuinely Better

```
1. DEVELOPER EXPERIENCE AND VELOCITY
   Workflow lives in the repo — no separate Jenkins server to access.
   PRs automatically get status checks. No webhook configuration.
   No plugin installation for standard tooling. New project gets
   CI in 5 minutes with a YAML file, not an hour configuring Jenkins.

2. ZERO OPERATIONAL BURDEN FOR CI COMPUTE
   No Jenkins master to maintain, patch, and keep available.
   No agent VMs to manage for standard builds.
   GitHub's SLA covers what you'd otherwise operate yourself.
   Platform teams can focus on actual product work, not CI operations.

3. SECURITY MODEL
   OIDC keyless auth is a step-change improvement over Jenkins'
```

```
   credential plugins. No long-lived cloud credentials stored anywhere.
   Ephemeral runners eliminate job cross-contamination by default.
   GitHub's audit log is comprehensive without plugin configuration.

4. GITHUB-NATIVE WORKFLOWS
   Pull request checks, deployment environments, required reviewers,
   PR annotations, GHCR package hosting, SLSA attestations —
   all built-in with zero configuration. Jenkins requires plugins
   and webhooks for each of these.

5. COST AT MODERATE SCALE
   For 1-50 developers with standard CI needs, GitHub-hosted runners
   are cheaper than operating Jenkins infrastructure including:
   EC2/GCE cost, ops engineer time, Jenkins licensing (enterprise),
   HA setup, backup systems.

6. SUPPLY CHAIN SECURITY
   SLSA provenance, SBOM generation, artifact attestation —
   all native via GitHub Actions toolchain. Jenkins requires
   external tooling and significant integration effort.

7. MARKETPLACE ECOSYSTEM MOMENTUM
   20,000+ Actions and growing fast. GitHub's ownership means
   first-party tooling (AWS, Azure, GCP, Docker, Kubernetes)
   gets Actions support simultaneously or before Jenkins plugins.
```

The Definitive Decision Framework

```
Choose GitHub Actions if ALL of these are true:
  ✓ Source code is on GitHub (or moving there)
  ✓ Runners can reach github.com (not fully air-gapped)
  ✓ Standard CI/CD patterns (build, test, deploy)
  ✓ Team is ≤ 500 developers with standard tools
  ✓ You want to reduce platform ops burden

Choose Jenkins if ANY of these are true:
  ✗ Source code is NOT on GitHub (GitLab, Bitbucket, on-prem git)
  ✗ Fully air-gapped / classified network (no github.com access)
  ✗ Pipelines require complex programmatic generation (Scripted Pipeline)
  ✗ Critical dependencies on Jenkins-only plugins with no GHA equivalent
  ✗ Mixed SCM environment where Jenkins is the single pane of glass
  ✗ On-prem hardware required for all compute (mainframe, ASIC tools)

Consider both if:
  → Large enterprise with diverse teams (some on GitHub, some on other
SCMs)
  → Transition period where migrating gradually
  → Some workloads fit GHA; some require Jenkins capabilities
```

Interview Answer (45-60 seconds)

> "The fundamental architectural difference is that Jenkins is a long-running server you operate, while GitHub Actions is an event-driven service GitHub manages. That single difference drives most of the tradeoffs.
>
> Jenkins is genuinely better in four situations: your code isn't on GitHub; you're air-gapped with no github.com access; you need complex programmatic pipeline generation that Groovy Scripted Pipelines enable naturally; or you have irreplaceable Jenkins-only plugin dependencies.
>
> GitHub Actions is better for everything else: zero operational burden on CI compute, native GitHub integration with PR checks and deployment environments, OIDC keyless cloud auth as a first-class feature, ephemeral runners by default for security isolation, and supply chain security features like SLSA provenance without external tooling.
>
> In a real conversation, I'd push back on the binary framing — most enterprises I've seen end up running both. Jenkins for the specialized workloads and legacy pipelines that genuinely need it, GitHub Actions for the new greenfield services and teams where developer experience and low ops burden are the priority."

Gotchas ⚠

- **"Jenkins is old and GitHub Actions is modern" is not an argument.** Jenkins is actively maintained (LTS releases), has a stable plugin ecosystem, and runs production CI at some of the world's largest companies. Age is not the comparison axis — capability and operational fit are.
- **GitHub Actions is NOT free at scale.** At 100,000+ CI minutes/month, the compute cost can exceed what equivalent self-managed Jenkins infrastructure would cost. The "free" narrative breaks down for large teams.
- **Migration cost is always underestimated.** Every Jenkins Shared Library function, every custom plugin integration, every team-specific pipeline pattern requires translation effort. Factor 3-6 months for a serious enterprise migration.
- **GHES (GitHub Enterprise Server) has a different feature set than GitHub.com.** Some GitHub Actions features (Larger Runners, some environment features) may lag on GHES. Always verify feature availability on the specific GHES version before committing to a capability in your migration plan.

10.2 Jenkins to GitHub Actions Migration Patterns

What It Is

The practical playbook for migrating CI/CD from Jenkins to GitHub Actions — including the conceptual mapping between the two systems, migration strategies (big-bang vs incremental), and how to handle the hard parts: Shared Libraries, complex pipeline logic, agent management, and credential migration.

The Conceptual Mapping — Jenkins to GHA

Every Jenkins concept maps to one or more GitHub Actions concepts. Knowing this mapping cold is essential for migration interviews.

```
JENKINS                           GITHUB ACTIONS
──────────────────────────────────────────────────────────────────────

Jenkins Master                    GitHub's orchestration service
                                   (you don't see or manage this)

Jenkinsfile                        .github/workflows/*.yml
                                   (one or more workflow files per repo)

Pipeline (Declarative/Scripted)    Workflow

Stage                              Job
  (stages run sequentially          (jobs run in parallel by default;
   within a pipeline)                use needs: for sequencing)

Step (within a stage)              Step (within a job)

agent { label 'X' }               runs-on: [self-hosted, X]
agent { docker 'image' }          container: image:tag
agent any                         runs-on: ubuntu-latest
agent none (at pipeline level)    (no equivalent — each job picks runner)

node('label') { ... }             jobs.<id>: runs-on: [self-hosted, label]

environment { KEY = 'val' }       env: KEY: val (job or step level)
                                   echo "KEY=val" >> $GITHUB_ENV (dynamic)

credentials('id')                  secrets.SECRET_NAME
                                   vars.VAR_NAME (non-secret)

withCredentials([...]) { }        env:
                                    MY_SECRET: ${{ secrets.MY_SECRET }}
                                   (scoped to step automatically)

parameters { string(...) }        on:
                                    workflow_dispatch:
                                      inputs:
                                        param_name:
                                          type: string

input('Approve deploy?')           environment: production
                                   (required reviewers configured in env)

when { branch 'main' }            if: github.ref == 'refs/heads/main'

when { changeset 'src/**' }       on:
                                    push:
                                      paths: ['src/**']
```

```
                                    OR dorny/paths-filter (8.2)

parallel { stage('a'){} }          strategy:
  stage('b'){} ...                    matrix:
                                        item: [a, b, ...]

Shared Library (function)           Composite Action
Shared Library (pipeline)           Reusable Workflow
Shared Library (vars object)        Workflow-level env: + vars context

stash / unstash                     actions/upload-artifact
                                    actions/download-artifact

archiveArtifacts                    actions/upload-artifact

junit testResults: '*.xml'          Upload artifact + third-party
                                    test reporter action or SARIF upload

post { always { } }                if: always() (on step or job)
post { failure { } }               if: failure()
post { success { } }               if: success() (same as default)

BUILD_NUMBER                        github.run_number
BUILD_ID                            github.run_id
GIT_COMMIT                          github.sha
GIT_BRANCH                          github.ref_name
JOB_NAME                            github.workflow
WORKSPACE                           github.workspace
                                     (also: $GITHUB_WORKSPACE env var)

Kubernetes plugin                   ARC (Actions Runner Controller)
Docker Pipeline plugin              docker/build-push-action + buildx
Slack Notification plugin           slackapi/slack-github-action
Email Extension plugin              actions/github-script (email via
SES/SendGrid)
```

Migration Strategy: Big-Bang vs Incremental

```
BIG-BANG MIGRATION:
  Approach: Convert all Jenkins pipelines to GitHub Actions at once
            Freeze Jenkins; cut over by a date; decommission after

  Timeline: 2-6 months (depending on pipeline complexity)

  Pros:
    - Clean cut — no dual-maintenance of two systems
    - Clearer milestone: "we're done"
    - Forces cleanup of legacy pipelines that nobody uses

  Cons:
```

```
    - High risk: everything fails at once if migration is incomplete
    - Requires significant upfront investment
    - Blockers (plugins with no equivalent) delay everything
    - Teams lose productivity during transition

  When to use:
    - Small org (< 20 engineers, < 50 pipelines)
    - New greenfield product starting on GitHub
    - Hard deadline (e.g., Jenkins license expiration, cost crisis)

──────────────────────────────────────────────────────────────────────

INCREMENTAL MIGRATION (Strangler Fig Pattern):
  Approach: Run Jenkins and GitHub Actions in parallel
            Migrate service by service or team by team
            Jenkins remains authoritative for unmigrated pipelines
            Decommission Jenkins only when nothing depends on it

  Timeline: 6-18 months (large org)

  Pros:
    - Each team migrates at their own pace
    - Failures are isolated (one service's GHA is broken, Jenkins still
works)
    - Can prioritize high-value/low-complexity pipelines first
    - Real-world learning before migrating hard cases

  Cons:
    - Dual-maintenance burden during transition
    - Engineers must know both systems
    - "Temporary" Jenkins pipelines become permanent
    - No clear decommission date without explicit tracking

  When to use:
    - Large org (50+ engineers, 200+ pipelines)
    - Diverse pipeline complexity (simple + very complex)
    - Risk-averse culture requiring gradual validation

RECOMMENDED APPROACH FOR MOST ORGs:
  Hybrid:
    Phase 1 (Month 1-2):   New services → GHA only
    Phase 2 (Month 2-6):   Migrate simple pipelines (no shared libraries)
    Phase 3 (Month 6-12):  Migrate medium complexity (shared libraries →
Actions)
    Phase 4 (Month 12+):   Tackle hard cases (complex dynamic pipelines)
    Phase 5 (Month 18):    Jenkins decommission (or keep for irremovable
cases)
```

Migrating Shared Libraries — The Hardest Part

Shared Libraries are the biggest migration complexity. They contain the institutional knowledge of your organization's build patterns.

```
Jenkins Shared Library Structure:
  shared-library/
    vars/                     # Global variables (callable as functions)
      deployService.groovy    # def call(Map config) { ... }
      buildDocker.groovy
    src/                      # Groovy classes
      org/myorg/Deploy.groovy
    resources/                # Static files
      scripts/setup.sh

How they're used:
  @Library('my-shared-lib') _
  deployService(
    service: 'payment-api',
    environment: 'production',
    image: "ghcr.io/my-org/payment-api:${BUILD_TAG}"
  )
```

```
MIGRATION MAPPING:

Shared Library FUNCTION (vars/*.groovy)
  -> Composite Action (.github/actions/<name>/action.yml)
  + Has inputs/outputs (matching Groovy function parameters)
  + Runs inline on the caller's runner
  + Can contain multiple shell steps
  - Cannot call other uses: (composite can in GHA v2)
  - No Groovy logic — shell script or JavaScript only

Shared Library PIPELINE TEMPLATE (src/ with full pipeline)
  -> Reusable Workflow (.github/workflows/<name>.yml)
  + Called at JOB level (not step level)
  + Can have its own jobs, matrix, environments
  + Supports secrets: inherit for credential passing
  - More rigid than Groovy (no dynamic stage generation)

Complex GROOVY LOGIC in shared libraries
  -> Custom GitHub Action (JavaScript or Docker)
  + Full programming language
  + Access to GitHub API via @actions/github toolkit
  - More development effort than a simple composite
  OR
  -> Shell script called from a composite action
  + Fast to implement
  - Limited to shell capabilities
```

Concrete Shared Library Migration Example

```
// BEFORE: Jenkins Shared Library (vars/deployService.groovy)
def call(Map config) {
    def service    = config.service
    def env        = config.environment ?: 'staging'
    def image      = config.image
    def replicas   = config.replicas ?: 2
    def namespace  = config.namespace ?: env

    stage("Deploy ${service} to ${env}") {
        withCredentials([
            string(credentialsId: "KUBECONFIG_${env.toUpperCase()}",
variable: 'KUBECONFIG_DATA')
        ]) {
            sh """
                echo "\$KUBECONFIG_DATA" > /tmp/kubeconfig
                helm upgrade --install ${service} ./helm/${service} \
                    --namespace ${namespace} \
                    --set image.tag=${image} \
                    --set replicaCount=${replicas} \
                    --wait --timeout 10m --atomic
                rm /tmp/kubeconfig
            """
        }
    }
}
```

```
# AFTER: GitHub Actions Composite Action
# .github/actions/deploy-service/action.yml

name: Deploy Service
description: Deploys a Helm chart to Kubernetes

inputs:
  service:
    description: 'Service name (must match helm/SERVICE directory)'
    required: true
  environment:
    description: 'Target environment'
    required: false
    default: 'staging'
  image-tag:
    description: 'Docker image tag to deploy'
    required: true
  replicas:
    description: 'Number of pod replicas'
    required: false
    default: '2'

outputs:
  helm-revision:
```

```
    description: 'Helm release revision number after deploy'
    value: ${{ steps.deploy.outputs.revision }}

runs:
  using: composite
  steps:
    - name: Configure kubectl
      shell: bash
      run: |
        # OIDC handles auth -- no stored kubeconfig needed
        aws eks update-kubeconfig \
          --name ${{ inputs.environment }}-cluster \
          --region us-east-1

    - name: Helm deploy
      id: deploy
      shell: bash
      run: |
        SERVICE="${{ inputs.service }}"
        ENV="${{ inputs.environment }}"
        IMAGE="${{ inputs.image-tag }}"
        REPLICAS="${{ inputs.replicas }}"

        helm upgrade --install "$SERVICE" "./helm/$SERVICE" \
          --namespace "$ENV" \
          --create-namespace \
          --set image.tag="$IMAGE" \
          --set replicaCount="$REPLICAS" \
          --wait \
          --timeout 10m \
          --atomic

        REVISION=$(helm history "$SERVICE" --namespace "$ENV" \
          --max 1 --output json | jq -r '.[0].revision')
        echo "revision=${REVISION}" >> $GITHUB_OUTPUT
```

```
# Usage in a workflow (equivalent to @Library call)
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment: production
    permissions:
      id-token: write
      contents: read
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683

      - uses: aws-actions/configure-aws-
credentials@e3dd6a429d7300a6a4c196c26e071d42e0343502
        with:
          role-to-assume: ${{ secrets.PROD_DEPLOY_ROLE_ARN }}
```

```
          aws-region: us-east-1

      # Equivalent to: deployService(service: 'payment-api', environment:
'production', image: '...', replicas: 3)
      - uses: ./.github/actions/deploy-service
        with:
          service: payment-api
          environment: production
          image-tag: sha-${{ github.sha }}
          replicas: '3'
```

Migrating Jenkins Credentials to GitHub Secrets

```
# STRATEGY: Map Jenkins credential types to GHA equivalents

# Jenkins Credential Type -> GitHub Actions Equivalent

# 1. Secret Text -> GitHub Secret
#    Jenkins: withCredentials([string(credentialsId: 'MY_TOKEN', variable:
'TOKEN')])
#    GHA:     env: MY_TOKEN: ${{ secrets.MY_TOKEN }}
gh secret set MY_TOKEN --body "$(jenkins_export_credential MY_TOKEN)" --
repo my-org/my-repo

# 2. Username/Password -> Two GitHub Secrets or one JSON secret
#    Jenkins: withCredentials([usernamePassword(credentialsId: 'DB_CREDS',
...)])
#    GHA:     DB_USERNAME + DB_PASSWORD as separate secrets
#    Or:      DB_CREDS as JSON string, parsed in workflow
gh secret set DB_USERNAME --body "myuser"
gh secret set DB_PASSWORD --body "mypassword"

# 3. SSH Private Key -> GitHub Secret (base64 encoded)
#    Jenkins: sshagent(['MY_SSH_KEY']) { sh 'git clone ssh://...' }
#    GHA:     Use webfactory/ssh-agent action or write key to file
gh secret set SSH_PRIVATE_KEY --body "$(cat ~/.ssh/id_rsa | base64)"

# 4. Kubeconfig / Cloud credentials -> OIDC (NO secret needed!)
#    Jenkins: withCredentials([file(credentialsId: 'KUBECONFIG')...])
#    GHA:     OIDC auth to cloud provider -> generate kubeconfig
dynamically
#    This is the most impactful improvement: no stored credentials at all

# 5. AWS Access Key / Secret -> OIDC (NO secret needed!)
#    Jenkins: withCredentials([aws(credentialsId: 'AWS_PROD')])
#    GHA:     aws-actions/configure-aws-credentials with role-to-assume
(OIDC)

# BULK MIGRATION SCRIPT (simplified)
# Export all Jenkins credentials (requires Jenkins admin + credentials-
```

```
binding plugin)
curl -u admin:${JENKINS_TOKEN} \
  "${JENKINS_URL}/credentials/store/system/domain/_/api/json" \
  | jq '.credentials[] | .id' | while read CRED_ID; do
    echo "Migrating: $CRED_ID"
    # ... export and import logic per credential type
  done
```

Migration Validation Pattern

```
# During migration, run BOTH Jenkins and GitHub Actions in parallel
# Compare results to validate parity before cutting over

name: Migration Validation

on: [push, pull_request]

jobs:
  gha-build:
    runs-on: ubuntu-latest
    outputs:
      test-result: ${{ steps.test.outputs.result }}
      artifact-hash: ${{ steps.hash.outputs.hash }}
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - id: test
        run: |
          npm test
          echo "result=passed" >> $GITHUB_OUTPUT
      - name: Build
        run: npm run build
      - name: Hash artifact
        id: hash
        run: |
          HASH=$(find dist/ -type f | sort | xargs sha256sum | sha256sum |
cut -d' ' -f1)
          echo "hash=${HASH}" >> $GITHUB_OUTPUT
      - uses: actions/upload-artifact@65c4c4a1ddee5b72f698c2d2b64e41
        with:
          name: gha-build-${{ github.sha }}
          path: dist/

  compare-with-jenkins:
    needs: gha-build
    runs-on: ubuntu-latest
    steps:
      - name: Fetch Jenkins build artifact
        run: |
          # Download the Jenkins build artifact for the same commit
          curl -u "${JENKINS_USER}:${JENKINS_TOKEN}" \
```

```
            "${JENKINS_URL}/job/my-app/lastBuild/artifact/dist.tar.gz" \
            -o jenkins-dist.tar.gz
          tar xzf jenkins-dist.tar.gz -C jenkins-dist/
        env:
          JENKINS_USER: ${{ secrets.JENKINS_USER }}
          JENKINS_TOKEN: ${{ secrets.JENKINS_TOKEN }}

      - uses: actions/download-
artifact@fa0a91b85d4f404e444e00e005971372dc801d16
        with:
          name: gha-build-${{ github.sha }}
          path: gha-dist/

      - name: Compare artifacts
        run: |
          # Compare key files between Jenkins and GHA builds
          diff <(find gha-dist/ -type f | sort) \
               <(find jenkins-dist/ -type f | sort) && \
            echo "File lists match" || \
            echo "WARNING: File lists differ"

          # Compare binary checksums
          GHA_HASH=$(find gha-dist/ -type f | sort | xargs sha256sum |
sha256sum)
          JENKINS_HASH=$(find jenkins-dist/ -type f | sort | xargs
sha256sum | sha256sum)

          if [ "$GHA_HASH" = "$JENKINS_HASH" ]; then
            echo "✅ Artifacts match -- GHA migration validated"
          else
            echo "⚠  Artifacts differ (may be expected for non-
deterministic builds)"
            echo "GHA hash:     $GHA_HASH"
            echo "Jenkins hash: $JENKINS_HASH"
          fi
```

The Hard Cases — What Doesn't Migrate Cleanly

```
1. SCRIPTED PIPELINE DYNAMIC STAGE GENERATION
   Jenkins:
     def services = getServicesFromDatabase()  // arbitrary code
     def stagesMap = [:]
     services.each { svc ->
       stagesMap["Deploy ${svc.name}"] = {
         stage("Deploy ${svc.name}") {
           // stage config from svc object
         }
       }
     }
     parallel stagesMap
```

```
   GHA approach:
     Approximation: pre-flight job queries DB, outputs JSON matrix,
     downstream job uses fromJSON() as matrix.
     Limitation: matrix is computed at job-start, not mid-pipeline.
     Cannot dynamically generate jobs from within a running job.

2. JENKINS INPUT STEP (mid-pipeline pause for human input)
   Jenkins:
     def answer = input(
       message: 'Deploy to production?',
       parameters: [choice(name: 'answer', choices: ['yes','no'])]
     )
     if (answer == 'yes') { deploy() }

   GHA approach:
     Environment protection rules pause BEFORE the job starts,
     not mid-job. Cannot ask mid-pipeline questions.
     Workaround: split pipeline into two jobs (pre-approval + post-
approval)
     with an environment gate between them.

3. COMPLEX PIPELINE INHERITANCE / OVERRIDE
   Jenkins: Shared Library base class extended by team pipelines.
   GHA: No inheritance. Reusable Workflows are called, not extended.
   Workaround: reusable workflow with many conditional inputs to
   cover all override scenarios. Gets unwieldy past ~10 variations.

4. BUILD RETENTION POLICIES
   Jenkins: Fine-grained retention per job (keep last N builds,
   keep builds for last X days, keep every Nth build permanently).
   GHA: Fixed retention per artifact (1-400 days). Less flexible.
   Workaround: custom cleanup workflows via API.

5. JENKINS VIEW CUSTOMIZATION
   Jenkins: Custom folder views, dashboard plugins, per-team landing
pages.
   GHA: Fixed GitHub Actions UI. Some customization via organization-level
   dashboards in GitHub Enterprise, but not equivalent.
```

Interview Answer (45-60 seconds)

> "The conceptual mapping is straightforward at the top level: Jenkinsfiles become workflow YAML files, stages become jobs, steps remain steps, agent labels become runner labels in `runs-on`. The hard part is the Shared Libraries.
>
> Shared Library functions that wrap step sequences map to Composite Actions — you translate the Groovy parameters to Action inputs, and the sh{} blocks become shell run steps. Shared Library pipeline templates map to Reusable Workflows — called at job level, run their own jobs with their own runners.

> For migration strategy on anything beyond a small org, I'd use the strangler fig pattern: new services go straight to GHA, existing services migrate incrementally by complexity tier — simple pipelines first, then medium, then the complex dynamic ones that require the most redesign. The credential migration is where OIDC provides a genuine architectural upgrade: instead of migrating AWS keys and kubeconfig credentials from Jenkins Credentials store to GitHub Secrets, you replace them entirely with OIDC trust relationships and eliminate the stored credentials completely.
>
> The genuinely hard cases are dynamic stage generation, mid-pipeline input steps, and complex pipeline inheritance patterns — these require rethinking the architecture, not just translating syntax."

Gotchas ⚠

- **Jenkins `currentBuild.result` has no direct equivalent.** In Jenkins you can read and set `currentBuild.result` mid-pipeline. In GHA, job result is read-only after the fact via `needs.<job>.result`. Design multi-step decision logic differently.
- **`post { always { } }` requires `if: always()` on EVERY cleanup step in GHA.** In Jenkins, `post { always {} }` covers the whole stage/pipeline. In GHA, each cleanup step independently needs `if: always()` — forgetting it means the cleanup doesn't run on failure.
- **Jenkins workspace is persistent per agent; GHA workspace is ephemeral per job.** Code that assumes a previous build left files on the agent (warm Docker cache, locally installed tools) will fail. Replace with explicit caching (`actions/cache`) and tool installation steps.
- **Jenkins Groovy DSL allows mid-stage logic; GHA steps are sequential and flat.** You can't write `if (something) { sh 'command' }` inline between steps in YAML. Use `if:` on steps or shell conditionals within a `run:` block.
- **The `BRANCH_NAME` variable in Jenkins is not the same as `github.ref_name` for PRs.** Jenkins' `BRANCH_NAME` for PR builds is the source branch (e.g., `feature/my-pr`). GHA's `github.ref_name` for `pull_request` events is the PR merge ref (e.g., `refs/pull/42/merge`). Use `github.head_ref` for the source branch in PR contexts.

10.3 GitHub Actions vs GitLab CI, CircleCI, Tekton

What It Is

A positioning comparison across the four CI/CD tools most commonly encountered in interview discussions: GitHub Actions, GitLab CI, CircleCI, and Tekton. The goal is not encyclopedic detail about each tool but the ability to articulate the key differentiation clearly and honestly.

The Four Tools at a Glance

```
GITHUB ACTIONS:    Event-driven, GitHub-native, serverless orchestration
GITLAB CI:         Deeply integrated with GitLab SCM; pipeline-as-code;
self-hostable
CIRCLECI:          Developer-focused SaaS CI; fast orbs ecosystem; strong
```

```
Docker support
TEKTON:            Kubernetes-native CI/CD framework; vendor-neutral CDF
project
```

GitHub Actions vs GitLab CI

Architecture Comparison

```
GitHub Actions                            GitLab CI
──────────────────────────────────────────────────────────────────────
SCM: GitHub only                          SCM: GitLab only
                                           (GitLab.com or self-managed)

Config file: .github/workflows/*.yml      Config file: .gitlab-ci.yml
  Multiple files allowed                    Single file (includes for reuse)
  Job: top-level concept                    Job: top-level concept

Compute:                                  Compute:
  GitHub-hosted (their VMs)                GitLab-hosted (SaaS runners)
  Self-hosted runners                      Self-managed runners (same agent
software)
  ARC (K8s operator)                       GitLab K8s integration (less
mature)

Reuse:                                    Reuse:
  Composite Actions (step-level)            include:
(file/template/component includes)
  Reusable Workflows (job-level)            extends: (job inheritance within
file)
  Actions Marketplace                       GitLab CI Components Catalog
(newer)

Secrets:                                  Secrets:
  Repo/Org/Environment scopes              Project/Group/Environment scopes
  OIDC (AWS, GCP, Azure, Vault)            OIDC (AWS, GCP, Azure, Vault)
  GitHub Secrets API                        GitLab Variables API

Security:                                 Security:
  OIDC keyless auth                         OIDC keyless auth (similar)
  SLSA provenance (native)                  SLSA via Sigstore integration
  Runner isolation options                  Similar runner isolation options

Environments:                             Environments:
  Protection rules (approval gates)         Deployment environments +
approvals
  Deployment history                        Deployment tracking
  Requires Team/Enterprise for approvals  Free on GitLab.com (with
limitations)

Package Registry:                         Package Registry:
```

```
  GHCR (containers)                        GitLab Container Registry
  GitHub Packages (npm, Maven, etc.)      GitLab Packages (npm, Maven,
etc.)

CD/GitOps:                              CD/GitOps:
  ArgoCD integration (external)           GitLab Agent for Kubernetes
(built-in)
  No built-in CD orchestration            GitLab Environments + Auto
DevOps
```

Key Differentiators

```
GitLab CI ADVANTAGES over GitHub Actions:
  1. Auto DevOps: zero-config CI/CD for Dockerized apps
     (detects language, builds, tests, deploys automatically)

  2. Built-in Kubernetes agent (GitLab Agent for K8s)
     Cluster management integrated with GitLab without external tools

  3. Single .gitlab-ci.yml with `extends:` for job inheritance
     More powerful template merging than GHA's composite actions

  4. Security scanning built into the platform (SAST, DAST, dependency
scan)
     as first-class pipeline stages, not marketplace actions

  5. Self-managed GitLab Server + GitLab Runner is fully air-gappable
     No github.com dependency at all

  6. MR (Merge Request) approval rules and pipeline gates are more
     granular and configurable than GHA's Environment protection

  7. Review Apps: deploy branch-specific environments on every MR
     GitHub has no direct equivalent

GitHub Actions ADVANTAGES over GitLab CI:
  1. GitHub marketplace has significantly more third-party actions
     (20,000+ vs GitLab's Component Catalog which is much smaller)

  2. GitHub's network effect: most open-source projects are on GitHub
     Consuming open-source actions is more natural

  3. Multiple workflow files (better separation of concerns)
     vs single .gitlab-ci.yml that grows very large

  4. Better PR/review integration (annotations, CODEOWNERS, status checks)

  5. Supply chain security (SLSA provenance, artifact attestations) is
     more mature and integrated

When to choose GitLab CI:
```

```
  - Already on GitLab (obviously)
  - Need Auto DevOps for fast onboarding
  - Want built-in security scanning without marketplace action management
  - Fully self-managed / air-gapped requirement (self-managed GitLab)
  - Need Review Apps feature
```

GitHub Actions vs CircleCI

Architecture Comparison

```
GitHub Actions                              CircleCI
──────────────────────────────────────────────────────────────────────────────
SCM: GitHub only                            SCM: GitHub, GitLab, Bitbucket
                                             (multi-SCM support)

Config: .github/workflows/*.yml             Config: .circleci/config.yml

Reuse:                                      Reuse:
  Composite Actions                           Orbs (reusable
commands/jobs/executors)
  Reusable Workflows                          Pipeline parameters
  Actions Marketplace                         Orb Registry (CircleCI)

Compute:                                    Compute:
  GitHub-hosted (ubuntu/mac/windows)          CircleCI Cloud (Docker-based
executors)
  Self-hosted runners / ARC                   Self-hosted (machine runners,
Runner)
  Larger runners (4-64 core)                  Resource classes (small to
2xlarge)
  ARM64 native                                ARM resource class (via self-
hosted)

Parallelism:                                Parallelism:
  Matrix strategy                             Parallelism key (split tests)
  Dynamic matrix                              Test splitting built-in
(powerful)
  Manual job splitting                        circleci tests split command
(native)

Caching:                                    Caching:
  actions/cache (manual keys)                 save_cache / restore_cache steps
  setup-* built-in cache                      persist_to_workspace (different
model)
  10 GB repo limit                            Cache TTL by workspace/branch

Docker:                                     Docker:
  docker/build-push-action                    Powerful Docker layer caching
  GHA cache backend                           DLC (Docker Layer Caching) --
premium
```

```
  Registry cache                          Remote Docker engine support

Environments:                           Environments:
  GitHub Environments + approval          Contexts (org-level secret
groups)
                                          Approval jobs
  No native "contexts" concept            Contexts map to runner groups +
env secrets

Pricing:                                Pricing:
  Minutes-based (free tier per plan)      Credit-based per compute minute
  Cheaper for Linux at scale with         Can be cheaper for Docker-heavy
workloads
  self-hosted ARC                         with DLC + parallel execution
```

Key Differentiators

```
CircleCI ADVANTAGES over GitHub Actions:
  1. Test splitting: circleci tests split --split-by=timings
     Native intelligent test distribution across parallel containers
     GHA has no equivalent (you'd implement manually via custom logic)

  2. Multi-SCM: works with GitHub, GitLab, Bitbucket in one account
     GHA is GitHub-only

  3. Docker Layer Caching (DLC): persistent Docker layer cache
     significantly faster Docker builds on repeat runs
     More reliable than GHA's type=gha cache for large images

  4. SSH debugging: native 'Rerun with SSH' button in UI
     GHA requires tmate action setup

  5. Resource classes: granular compute selection
(nano/small/medium/large/xlarge/2xlarge)
     Fine-grained cost optimization per job
     GHA has 2-core standard + specific larger runner sizes

  6. Workflows are DAG-based by default with rich visualization
     Better visual pipeline representation

GitHub Actions ADVANTAGES over CircleCI:
  1. GitHub-native: no webhook setup, no OAuth app, direct PR integration
  2. Free compute (included minutes for GitHub-hosted)
  3. OIDC keyless cloud auth is first-class
  4. GitHub Environments for approval gates (free tier has limits, but
integrated)
  5. Actions Marketplace is larger
  6. Supply chain security tooling is ahead

When to choose CircleCI:
  - Multi-SCM environment (GitHub + GitLab + Bitbucket)
```

```
  - Heavy Docker build workload where DLC ROI is significant
  - Need sophisticated test splitting by timing history
  - Already invested in CircleCI orbs
  - Need native SSH debugging without custom action setup
```

GitHub Actions vs Tekton

Architecture Comparison

```
GitHub Actions                            Tekton
──────────────────────────────────────────────────────────────────────

Type: SaaS + self-hosted hybrid           Type: Kubernetes-native framework
                                           (open source, runs in YOUR
cluster)

Governance: GitHub (vendor)               Governance: CNCF / CDF (vendor-
neutral)

SCM coupling: GitHub required             SCM coupling: None (integrates
with any)
                                           (GitLab, GitHub, Bitbucket,
Gerrit)

Config: YAML workflow files in repo       Config: Kubernetes CRDs
  Events trigger workflows                (Task, Pipeline, PipelineRun,
EventListener)
                                           Tekton Triggers for event-driven

Compute:                                  Compute:
  GitHub-hosted VMs                        Kubernetes pods (your cluster)
  Self-hosted runners                      All compute is YOUR K8s cluster
  ARC (K8s operator) -- closest analog     More control; more operational
burden

Reuse:                                    Reuse:
  Composite Actions                          Tasks (reusable step sequences)
  Reusable Workflows                         Pipelines (reusable pipeline
templates)
  Actions Marketplace                        Tekton Hub (community catalog)

UI:                                       UI:
  GitHub Actions UI (integrated)             Tekton Dashboard (self-hosted)
                                             OR Tekton in OpenShift (Red Hat)
                                             Backstage plugin for larger orgs

Secret management:                        Secret management:
  GitHub Secrets                             Kubernetes Secrets
                                             External secrets operators
(Vault, etc.)
```

```
Event triggers:                          Event triggers:
  GitHub webhooks (native)                Tekton Triggers + EventListeners
                                           Any webhook (GitHub, GitLab,
generic HTTP)
```

Key Differentiators

```
Tekton ADVANTAGES over GitHub Actions:
  1. Vendor neutral: no lock-in to GitHub, no github.com dependency
     Runs entirely in your Kubernetes cluster
     SCM agnostic: works with any git host

  2. True Kubernetes-native: pipeline steps ARE container specs
     Full Kubernetes resource model (resources, limits, volumes, sidecars)
     First-class integration with K8s RBAC, NetworkPolicy, PSA

  3. Air-gapped and on-premises: works with no external network access
     Control plane is YOUR cluster, not a vendor's service

  4. Task / Pipeline reuse is first-class with versioned references
     Community Tasks in Tekton Hub

  5. Better fit for Kubernetes-centric organizations
     CI/CD is part of the platform, not a separate service

  6. No per-minute billing from vendor
     Cost = your K8s cluster compute cost only

  7. Foundation for enterprise products:
     Red Hat OpenShift Pipelines = Tekton (enterprise support)
     JFrog Pipelines, etc. build on Tekton

GitHub Actions ADVANTAGES over Tekton:
  1. Dramatically lower operational burden
     Tekton requires K8s cluster management, CRD installation,
     Tekton Dashboard, Triggers setup -- significant ops investment
     GHA is zero ops for hosted compute

  2. Developer experience: YAML in the repo, not K8s CRDs
     Tekton PipelineRun YAML is much more verbose

  3. GitHub PR integration: status checks, annotations, CODEOWNERS --
     all native in GHA. Tekton requires custom integration work.

  4. Marketplace: 20,000+ Actions vs Tekton Hub (hundreds of Tasks)

  5. OIDC cloud auth: first-class in GHA. Tekton needs custom config.

  6. No cluster to manage for compute (GitHub-hosted runners)

When to choose Tekton:
```

```
  - Kubernetes-only shop where all workloads run in K8s
  - Vendor-neutral requirement (no GitHub dependency)
  - Air-gapped / classified environment
  - Multi-SCM org where GitHub lock-in is unacceptable
  - OpenShift users (OpenShift Pipelines = Tekton with OCP integration)
  - Building a platform team product on top of a CI/CD framework
```

Quick Positioning Matrix

```
                    GitHub      GitLab      CircleCI    Tekton
                    Actions     CI
____________________________________________________________________

GitHub-native         ✅✅        ❌          ✅          ❌
Multi-SCM             ❌          ❌          ✅          ✅
Air-gapped            ⚠           ✅          ❌          ✅
Hosted compute        ✅          ✅          ✅          ❌  (yours)
K8s-native            ⚠ (ARC)     ⚠           ❌          ✅✅
Zero vendor lock-in   ❌          ❌          ❌          ✅
Marketplace size      ✅✅        ⚠           ✅          ⚠
PR integration        ✅✅        ✅ (MR)     ✅          ⚠
Security scanning     ⚠           ✅✅        ⚠           ❌
Test splitting        ⚠           ⚠           ✅✅        ⚠
Auto DevOps           ❌          ✅✅        ❌          ❌
Review Apps           ❌          ✅          ❌          ❌
OIDC auth             ✅✅        ✅          ✅          ⚠
Operational burden    Low         Low         Low         High
```

Interview Answer (30-45 seconds)

> "The quick positioning: GitHub Actions for GitHub-native workloads with zero ops burden. GitLab CI for GitLab-native workloads — plus it has genuine advantages in built-in security scanning, Review Apps, and Auto DevOps that GHA doesn't match. CircleCI's differentiators are multi-SCM support, native test splitting by timing history, and Docker Layer Caching for heavy Docker workloads. Tekton is the vendor-neutral, air-gapped-capable, Kubernetes-native choice — it has no github.com dependency and runs entirely in your cluster, making it the right call for classified environments or organizations with hard vendor lock-in requirements.
>
> The honest answer for most GitHub-native teams is: GitHub Actions wins on developer experience and ecosystem unless you have a specific capability gap — CircleCI for test splitting, GitLab CI if you're on GitLab, Tekton if you need true vendor neutrality or air-gap."

10.4 Enterprise GitHub Actions Governance

What It Is

Governance is the set of policies, controls, and processes that ensure GitHub Actions is used safely, consistently, and in compliance with organizational security and operational requirements across all repositories in an organization or enterprise. Without it, every team invents their own patterns, security posture becomes uneven, and incidents proliferate.

The Governance Problem at Scale

```
Ungoverned org (100 repos, 50 developers):
  - 30 different patterns for Docker builds
  - 15 different credential management approaches (some using plaintext!)
  - 10 repos using self-hosted runners on public repos
  - 6 repos with pull_request_target and unchecked fork code execution
  - 3 repos pinning actions to SHA; 47 using floating tags (supply chain
risk)
  - No consistent notification on deployment failure
  - No audit of who ran what on which runner

Governed org:
  - Platform team owns the runner fleet and deployment patterns
  - All Docker builds use the organization's approved multi-arch build
action
  - OIDC is the only approved cloud auth method (no long-lived keys)
  - Required workflow enforces security scan on every PR
  - CODEOWNERS requires platform review for workflow file changes
  - Runner groups restrict production runners to specific repos
  - Weekly audit log review is automated
  - Drift is detected and alerted within 24 hours
```

Layer 1: Organization Action Policies

The first and coarsest control: which Actions can run in your org at all.

```
GitHub Org Settings -> Actions -> General -> Actions Permissions

Options:
  Allow all actions and reusable workflows
    -> No restriction. Any action from any source is permitted.
    -> Appropriate only for dev/personal orgs

  Allow enterprise actions and reusable workflows
    -> Only actions from within your GitHub Enterprise
    -> Too restrictive for most — blocks GitHub-owned actions

  Allow GitHub-owned actions and reusable workflows
    -> Only actions from the github org itself (e.g., actions/checkout)
```

```
    -> Very restrictive — blocks docker/*, aws-actions/*, etc.

  Allow GitHub-owned + select actions (RECOMMENDED for enterprises)
    -> GitHub-owned actions allowed (actions/checkout, etc.)
    -> Verified creator actions (docker, aws-actions, azure, google-
github-actions)
    -> Specific patterns you approve (my-org/*, trusted-partner/*)
```

```
# Setting via API (requires org admin token)
# PUT /orgs/{org}/actions/permissions/workflow

# Allow only GitHub-owned + verified + specific patterns:
gh api --method PUT /orgs/my-org/actions/permissions \
  -H "Accept: application/vnd.github+json" \
  -f enabled_repositories=all \
  -f allowed_actions=selected

gh api --method PUT /orgs/my-org/actions/permissions/selected-actions \
  -H "Accept: application/vnd.github+json" \
  -F github_owned_allowed=true \
  -F verified_allowed=true \
  -f patterns_allowed='["my-org/*", "docker/*@v*", "aws-actions/*@v*"]'
```

Layer 2: Required Workflows (GitHub Enterprise)

Required workflows are organization-level workflows that run on ALL pull requests to ALL (or selected) repositories in the org — repositories cannot opt out, and their checks must pass for merges to be allowed.

```
# In my-org/.github/workflows/required-security-scan.yml
# This workflow runs on EVERY PR in EVERY repo in the org

name: Required Security Scan

on:
  pull_request:
    branches: ['**']

permissions:
  contents: read
  security-events: write
  pull-requests: read

jobs:
  # ── 1. Enforce action pinning ──────────────────────────────────
  check-action-pinning:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
```

```
      - name: Check all actions are SHA-pinned
        run: |
          VIOLATIONS=()

          # Find all workflow files including changed ones
          while IFS= read -r WORKFLOW; do
            # Find uses: lines that are NOT pinned to a SHA
            while IFS= read -r LINE; do
              # Match uses: with a tag (@v1, @main, @latest) but not SHA
              if echo "$LINE" | grep -qE '^\s+uses:\s+[^#]+@(?!([0-9a-f]
{40}))'; then
                ACTION=$(echo "$LINE" | grep -oE '[a-zA-Z0-
9_/-]+@[^[:space:]#]+')
                VIOLATIONS+=("$WORKFLOW: $ACTION")
              fi
            done < <(grep -n 'uses:' "$WORKFLOW" 2>/dev/null)
          done < <(find .github/workflows -name '*.yml' -o -name '*.yaml')

          if [ ${#VIOLATIONS[@]} -gt 0 ]; then
            echo "::error title=Action Pinning::Found actions not pinned
to SHA:"
            for V in "${VIOLATIONS[@]}"; do
              echo "::error::$V"
            done
            echo ""
            echo "All third-party actions must be pinned to a full SHA
commit hash."
            echo "Example: uses:
actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683  # v4"
            exit 1
          fi
          echo "✅ All actions are properly SHA-pinned"

  # ── 2. Detect pull_request_target usage ──────────────────────
  check-dangerous-triggers:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - name: Check for dangerous trigger patterns
        run: |
          FOUND=false

          # Check for pull_request_target
          if grep -r 'pull_request_target' .github/workflows/ 2>/dev/null;
then
            echo "::warning title=Security Review
Required::pull_request_target trigger detected"
            echo "::warning::This trigger runs with repository secrets and
is dangerous with fork PRs."
            echo "::warning::Requires explicit platform team review."
            FOUND=true
          fi
```

```
          # Check for workflow_run without conclusion check
          if grep -rA5 'workflow_run:' .github/workflows/ 2>/dev/null | \
             grep -qE 'uses:|run:'; then
            echo "::notice::workflow_run trigger detected — ensure
conclusion is checked before running"
          fi

          # Dangerous: checking out fork PR head in pull_request_target
context
          if grep -l 'pull_request_target' .github/workflows/* 2>/dev/null
| \
             xargs grep -l 'head.sha\|head_ref' 2>/dev/null; then
            echo "::error title=Critical Security Issue::Checkout of fork
PR head SHA in pull_request_target context"
            echo "::error::This allows fork code to run with repository
secrets. Immediate review required."
            exit 1
          fi

  # ── 3. Validate secret usage patterns ──────────────────────────
  check-secret-patterns:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - name: Check secret handling patterns
        run: |
          # Detect secrets being echoed (potential leak)
          if grep -rn 'echo.*secrets\.' .github/workflows/ 2>/dev/null;
then
            echo "::warning title=Secret Leak Risk::Found echo of secret
value"
            echo "::warning::Secrets in echo statements appear in logs.
Use add-mask or avoid logging."
          fi

          # Detect script injection patterns
          if grep -rn 'run:.*\${{.*github\.event' .github/workflows/
2>/dev/null; then
            echo "::error title=Script Injection Risk::Untrusted input
used directly in run: step"
            echo "::error::Use env: to pass github.event.* values, not
direct ${{ }} interpolation"
            exit 1
          fi

  # ── 4. Dependency review ───────────────────────────────────────
  dependency-review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683
      - uses: actions/dependency-review-
action@e0df4cc0d2516fc0c9a9b28aae7e01bbc9c52c31
        with:
          fail-on-severity: high
```

```
          deny-licenses: GPL-3.0, AGPL-3.0

  # ── 5. Gate job (always posts status check) ──────────────────
  security-gate:
    needs: [check-action-pinning, check-dangerous-triggers, check-secret-
patterns, dependency-review]
    if: always()
    runs-on: ubuntu-latest
    steps:
      - run: |
          for RESULT in \
            "${{ needs.check-action-pinning.result }}" \
            "${{ needs.check-dangerous-triggers.result }}" \
            "${{ needs.check-secret-patterns.result }}" \
            "${{ needs.dependency-review.result }}"; do
            [ "$RESULT" = "failure" ] && echo "Security gate FAILED" &&
exit 1
          done
          echo "Security gate PASSED"
```

Layer 3: CODEOWNERS for Workflow Files

```
# .github/CODEOWNERS
# Require platform team review for ANY workflow change

.github/workflows/     @my-org/platform-team
.github/actions/       @my-org/platform-team

# Specific high-risk workflows require senior platform review
.github/workflows/deploy-production.yml    @my-org/platform-leads
.github/workflows/release.yml              @my-org/platform-leads

# Required workflows (only platform team creates these)
# Stored in .github repo at org level -- not in individual repos
```

Layer 4: Runner Group Access Control

```
# ── Restrict production runners to specific repos ──────────────

# Create production runner group (org-level)
gh api --method POST /orgs/my-org/actions/runner-groups \
  -f name="production-runners" \
  -f visibility="selected" \
  -F allows_public_repositories=false

# Get the group ID
GROUP_ID=$(gh api /orgs/my-org/actions/runner-groups \
```

```
  --jq '.runner_groups[] | select(.name == "production-runners") | .id')

# Grant only specific repos access to production runners
# (payment-api and auth-service are the only repos that can deploy)
PAYMENT_REPO_ID=$(gh api /repos/my-org/payment-api --jq '.id')
AUTH_REPO_ID=$(gh api /repos/my-org/auth-service --jq '.id')

gh api --method PUT \
  /orgs/my-org/actions/runner-groups/$GROUP_ID/repositories \
  -F "repository_ids[]=$PAYMENT_REPO_ID" \
  -F "repository_ids[]=$AUTH_REPO_ID"

# Register runners into the production group only
# (done during runner setup with --runnergroup flag)
```

Layer 5: Organization-Level Secrets and Variables

```
# ── Org-level secrets (available to selected repos) ──────────

# Create org secrets with selected visibility
gh secret set SHARED_NPM_TOKEN \
  --org my-org \
  --body "npm_token_value" \
  --visibility selected \
  --repos "my-org/service-a,my-org/service-b,my-org/service-c"

# Org-level variables (non-sensitive, visible in logs)
gh variable set ORG_REGISTRY \
  --org my-org \
  --body "ghcr.io/my-org" \
  --visibility all

gh variable set SLACK_CHANNEL \
  --org my-org \
  --body "#deployments" \
  --visibility all

# ── Environment secrets (most restricted) ────────────────────
# Create production environment with required reviewers
gh api --method PUT \
  /repos/my-org/payment-api/environments/production \
  --field wait_timer=0 \
  --field prevent_self_review=true

# Set environment-scoped secrets (only accessible when job targets the
env)
gh secret set PROD_DATABASE_URL \
  --repo my-org/payment-api \
  --env production \
  --body "postgres://prod-db:5432/payments"
```

Layer 6: Org-Level Reusable Workflow Library

```
# Structure: dedicated repo for org-wide reusable workflows
my-org/.github/
  workflow-templates/          # Template files for creating new repos
    nodejs-ci.yml
    go-service.yml
    python-service.yml

my-org/platform-workflows/     # Reusable workflows (platform team
manages)
  .github/
    workflows/
      docker-build.yml         # Standard multi-arch Docker build
      helm-deploy.yml          # Standard Helm deployment
      terraform-plan-apply.yml # Standard Terraform workflow
      security-scan.yml        # Standard security scanning
      notify-slack.yml         # Standard Slack notification

# Teams reference platform workflows:
jobs:
  build:
    uses: my-org/platform-workflows/.github/workflows/docker-build.yml@v2
    with:
      service-name: payment-api
    secrets: inherit

  deploy:
    uses: my-org/platform-workflows/.github/workflows/helm-deploy.yml@v2
    with:
      environment: production
      image-tag: ${{ needs.build.outputs.image-tag }}
    secrets: inherit
```

Layer 7: Automated Governance Audit

```yaml
# Weekly governance health check across all org repos
name: Org Governance Audit

on:
  schedule:
    - cron: '0 9 * * 1'    # Monday 9 AM
  workflow_dispatch:

permissions:
  contents: read
```

```
jobs:
  audit:
    runs-on: ubuntu-latest
    steps:
      - name: Audit all org repos for governance compliance
        run: |
          VIOLATIONS=()

          # Get all repos
          REPOS=$(gh api /orgs/my-org/repos --paginate --jq '.[].name')

          for REPO in $REPOS; do
            # Check: Does the repo use required workflow? (skip --
enforced by GH)

            # Check: Are workflow files in the repo using SHA-pinned
actions?
            WORKFLOWS=$(gh api /repos/my-
org/$REPO/contents/.github/workflows \
              --jq '.[].name' 2>/dev/null || echo "")

            for WF in $WORKFLOWS; do
              CONTENT=$(gh api \
                /repos/my-org/$REPO/contents/.github/workflows/$WF \
                --jq '.content' | base64 -d 2>/dev/null || echo "")

              # Check for floating tags
              if echo "$CONTENT" | grep -qE 'uses:.*@(v[0-
9]|main|latest|master)'; then
                VIOLATIONS+=("$REPO/$WF: unpinned action reference")
              fi

              # Check for secrets in echo
              if echo "$CONTENT" | grep -qE 'echo.*\$\{\{.*secrets\.';
then
                VIOLATIONS+=("$REPO/$WF: potential secret echo")
              fi

              # Check for pull_request_target
              if echo "$CONTENT" | grep -q 'pull_request_target'; then
                VIOLATIONS+=("$REPO/$WF: pull_request_target usage (needs
review)")
              fi
            done
          done

          # Report
          cat >> $GITHUB_STEP_SUMMARY << EOF
          ## Weekly Governance Audit

          **Repos scanned:** $(echo "$REPOS" | wc -l)
          **Violations found:** ${#VIOLATIONS[@]}
```

```
          $(if [ ${#VIOLATIONS[@]} -gt 0 ]; then
            echo "### Violations"
            for V in "${VIOLATIONS[@]}"; do
              echo "- \`$V\`"
            done
          else
            echo "✅ No violations found"
          fi)
          EOF

          # Create issue if violations found
          if [ ${#VIOLATIONS[@]} -gt 0 ]; then
            BODY=$(printf '%s\n' "${VIOLATIONS[@]}" | \
              jq -R . | jq -s . | \
              jq -r 'map("- `\(.)`") | join("\n")')

            gh api --method POST /repos/my-org/.github/issues \
              -f title="[Governance] Weekly audit found ${#VIOLATIONS[@]}
violations" \
              -f body="## Governance Violations\n\n${BODY}\n\nPlease
remediate." \
              -f labels='["governance", "security"]'
          fi
        env:
          GH_TOKEN: ${{ secrets.ORG_AUDIT_TOKEN }}
```

Governance Maturity Model

```
LEVEL 1 — Ad-hoc (no governance):
  □ No org action policy (all actions allowed)
  □ No CODEOWNERS on workflow files
  □ Credentials stored as individual repo secrets with no standardization
  □ No required workflows
  □ Self-hosted runners unrestricted

LEVEL 2 — Basic controls:
  ■ Org action policy: GitHub-owned + verified + curated allow-list
  ■ CODEOWNERS: platform team required for .github/** changes
  ■ Runner groups: production runners restricted to specific repos
  □ No required workflows
  □ Credential management is inconsistent

LEVEL 3 — Standard governance:
  ■ All Level 2 controls
  ■ Required workflow: security scan on all PRs
  ■ OIDC is standard for cloud auth (no long-lived credentials)
  ■ Org-level reusable workflow library in platform-workflows repo
  ■ Platform-owned workflow templates for common patterns
  ■ Monthly manual audit review
```

```
LEVEL 4 — Enterprise governance:
  ■ All Level 3 controls
  ■ Automated weekly governance audit workflow
  ■ Audit log streaming to SIEM (Splunk/Datadog)
  ■ Required workflow + required reviewers on environment gates
  ■ Ephemeral ARC runners for all compute (no persistent runner state)
  ■ Dependency Review action on all PRs
  ■ SHA-pinning enforced by required workflow scan
  ■ Incident runbooks for common GHA security scenarios
  ■ Quarterly red team exercises on runner security

LEVEL 5 — Mission-critical:
  ■ All Level 4 controls
  ■ SLSA Level 3 provenance on all build artifacts
  ■ Artifact attestation verification before every deploy
  ■ Network isolation (NetworkPolicy) on all runner pods
  ■ Immutable audit log storage (S3 Object Lock)
  ■ Automated rollback triggered by audit anomalies
  ■ SOC 2 / FedRAMP control mapping documented
```

Interview Answer (45-60 seconds)

> "Enterprise GitHub Actions governance operates in layers. The first layer is the org action policy — restricting to GitHub-owned actions, verified creator actions, and an approved allow-list of specific patterns. This prevents teams from pulling arbitrary code from the Marketplace without review.
>
> The second layer is Required Workflows on GitHub Enterprise: a platform team-owned workflow in the `.github` repo runs on EVERY PR in EVERY repository. It enforces SHA pinning of actions, detects dangerous patterns like `pull_request_target` with fork code checkout, and blocks script injection via `${{ github.event.* }}` in run steps.
>
> The third layer is CODEOWNERS: any change to `.github/workflows/` requires platform team review before merge. This means no team can add or modify a workflow without platform oversight.
>
> The fourth layer is runner group access control: production runners are restricted to only the repos that need deploy access via the runner groups API. A rogue workflow in an unrelated repo cannot use the production runner.
>
> The operational layer is an automated weekly audit workflow that scans all org repos for governance violations — unpinned actions, potential secret echoes, dangerous triggers — and creates a GitHub Issue with the findings. This catches drift between formal reviews."

Gotchas ⚠

- **Required Workflows on GHES may lag behind GitHub.com feature releases.** Always verify the specific GHES version supports Required Workflows before including it in your governance plan. Required Workflows was introduced in GHES 3.10.

- **CODEOWNERS doesn't prevent direct pushes to the protected branch.** CODEOWNERS requires pull request review — it doesn't block `git push --force` by a repo admin. Combine with branch protection (require PR, require review, include admins in restrictions).
- **Org action policies don't apply to self-hosted runners using `act` locally.** Policies are enforced by the GitHub orchestration service. Local `act` runs bypass them entirely. This is fine for development but means you can't test policy enforcement locally.
- **Required Workflows count toward the branch protection required status checks.** If a Required Workflow fails, the PR is blocked — even if the repo-level branch protection has no required checks. This is the desired behavior, but teams are surprised when a "new" required check appears on all their PRs after the platform team enables a Required Workflow.
- **Runner groups with `visibility: selected` must be explicitly updated when new repos are created.** New repos don't automatically get access to restricted runner groups. Automate runner group access grants as part of your new-repo provisioning process (GitHub webhook on `repository.created` event triggering an update).
- **Governance at scale requires a GitHub App, not an org member PAT.** PATs are tied to individual humans. When that person leaves, all governance automation breaks. Use a GitHub App with minimum required permissions for all governance automation.

Cross-Topic Interview Questions

Q: Your organization has 300 developers, 200 repos, uses Jenkins today, and wants to move to GitHub Actions over 18 months. You're the platform engineer leading this. What's your plan?

> "Eighteen months, 200 repos — this is an incremental migration. I'd structure it in five phases.
>
> Phase 1 (Months 1-2): Foundation. Set up the GitHub Actions infrastructure before migrating anything: org action policy (approved allow-list), runner fleet (ARC on K8s spot nodes for cost), OIDC trust relationships with AWS and GCP, the platform-workflows repo with the six core reusable workflows (Docker build, Helm deploy, Terraform, security scan, npm build, Go build). Get the governance layer in place: CODEOWNERS, required security scan workflow. No migrations yet — just building what teams will migrate TO.
>
> Phase 2 (Months 2-6): Easy wins. Identify the ~80 repos with simple pipelines — no shared libraries, standard builds. Migrate these first. Each migration takes 2-4 hours. Run GHA and Jenkins in parallel for 2 weeks to validate parity. Track: CI time before/after, failure rate, developer satisfaction.
>
> Phase 3 (Months 6-12): Shared library migration. Map all Shared Library functions to Composite Actions, pipeline templates to Reusable Workflows. This is the hardest part. Replace credentials with OIDC. Migrate ~90 medium-complexity repos. Some will need architectural changes (dynamic pipeline generation, mid-pipeline input steps).
>
> Phase 4 (Months 12-16): Hard cases. The 30 repos with complex dynamic pipelines, specialized plugins, or exotic agent requirements. Some of these may never fully migrate — identify early and categorize as 'Jenkins forever' if the ROI isn't there.

> Phase 5 (Months 16-18): Decommission. Once no repo has active Jenkins pipelines, decommission the Jenkins master. Keep a read-only instance for historical build records for 90 days.
>
> Metrics tracked throughout: migration velocity (repos/week), CI time improvement, credential management improvement (% using OIDC vs stored secrets), developer satisfaction (quarterly survey)."

Q: A team wants to use a third-party GitHub Action from the Marketplace that's not on your org's approved allow-list. What's the process?

> "We have a lightweight approval process: the team submits a PR to the platform team's allow-list configuration repo, including the action name, pinned SHA, a link to the action's source code, a brief security review (does it require excessive permissions? does it make external network calls to unknown endpoints? who maintains it? how many stars, how recently updated?), and the business justification.
>
> Platform team reviews within 48 hours. Approval criteria: the action must be from a verified creator or have a strong reputation and active maintenance; it must not request permissions beyond what's necessary; the source code should be auditable. If approved, the allow-list pattern is updated (e.g., add `my-trusted-org/their-action@*` to the patterns), and the team is free to use it.
>
> The action must be pinned to a SHA in their workflow — not a tag. We enforce this via the required workflow scan. If they need to update the action version, they update the SHA and the PR triggers a re-review only if the new SHA introduces significant changes.
>
> For actions with especially broad permissions or network access, we may require periodic re-review (quarterly) and will revoke allow-list access if the action's security posture degrades."

Q: How would you convince a skeptical CTO that migrating from Jenkins to GitHub Actions is worth the 12-18 month investment?

> "I'd frame it around four business outcomes, not technical features.
>
> First, engineering velocity. Today, every new service needs a Jenkins pipeline configured — coordination with the platform team, webhook setup, shared library onboarding. With GHA, a developer adds a YAML file to their repo and CI works. Time-to-first-CI for a new service goes from 3 days to 3 hours. At 300 developers launching a dozen services a year, that's meaningful acceleration.
>
> Second, operational cost. Jenkins requires dedicated platform engineers to keep the master running, manage plugins, perform upgrades, and on-call for Jenkins outages. GitHub handles the orchestration service. We redirect those engineering hours toward product work. Plus, with ARC on spot Kubernetes, compute costs drop 60-70% compared to on-demand Jenkins agents.
>
> Third, security posture. Jenkins credentials stores contain long-lived cloud credentials that rotate manually and are frequently forgotten. OIDC eliminates stored credentials entirely — no credential to rotate, no credential to leak. Ephemeral runners eliminate the cross-job contamination risk that persistent Jenkins agents carry. This directly reduces our attack surface.

> Fourth, talent. 'We use GitHub Actions' is a recruiting asset in 2025. Senior engineers at product companies expect modern CI/CD. 'We still run Jenkins' is a yellow flag for some candidates.
>
> The investment is 12-18 months of migration work. The payoff is permanent: reduced ops burden, better security, faster onboarding, lower compute cost."

Quick Reference Cheat Sheet

Jenkins → GitHub Actions Mapping

```
Jenkins                      GitHub Actions
──────────────────────────────────────────────────────
Jenkinsfile                  .github/workflows/*.yml
Pipeline                     Workflow
Stage                        Job (parallel by default)
Step                         Step (sequential)
agent { label 'X' }          runs-on: [self-hosted, X]
post { always {} }           if: always()
parameters { }               on: workflow_dispatch: inputs:
input('Approve?')            environment: production (approval gate)
stash/unstash                upload-artifact/download-artifact
archiveArtifacts             actions/upload-artifact
Shared Library function      Composite Action
Shared Library pipeline      Reusable Workflow
BUILD_NUMBER                 github.run_number
GIT_COMMIT                   github.sha
BRANCH_NAME                  github.ref_name (or github.head_ref for PRs)
```

Tool Selection Decision Tree

```
Is your source code on GitHub?
  NO  -> Jenkins (if multi-SCM) or GitLab CI (if on GitLab)
  YES -> Continue

Is the environment air-gapped (no github.com access)?
  YES -> Tekton (or Jenkins) -- GHA needs github.com
  NO  -> Continue

Do you need complex programmatic pipeline generation?
  YES -> Evaluate Jenkins Scripted Pipeline vs GHA workarounds
  NO  -> Continue

Is multi-SCM support required (GitHub + GitLab + Bitbucket)?
  YES -> CircleCI or Jenkins
  NO  -> GitHub Actions

Do you need native test splitting by timing history?
```

```
    YES -> Consider CircleCI as well
    NO  -> GitHub Actions

  → Default recommendation: GitHub Actions
```

Governance Checklist

```
  Layer 1 — Org Policy:
    [ ] Action policy: GitHub-owned + verified + curated allow-list
    [ ] Default workflow permissions: read-only GITHUB_TOKEN

  Layer 2 — Required Workflows (Enterprise):
    [ ] Security scan required on all PRs
    [ ] SHA-pinning enforcement
    [ ] pull_request_target + fork checkout detection

  Layer 3 — CODEOWNERS:
    [ ] .github/workflows/ -> @platform-team
    [ ] .github/actions/   -> @platform-team

  Layer 4 — Runner Control:
    [ ] Runner groups: production runners scoped to deploy repos
    [ ] allows_public_repositories: false on all runner groups

  Layer 5 — Secrets:
    [ ] OIDC for all cloud auth (no stored credentials)
    [ ] Org secrets: visibility=selected, not all
    [ ] Environment secrets for prod credentials

  Layer 6 — Platform Library:
    [ ] platform-workflows repo with versioned reusable workflows
    [ ] Workflow templates in .github repo

  Layer 7 — Audit:
    [ ] Weekly automated governance audit workflow
    [ ] Audit log streaming to SIEM
    [ ] Required workflow for dependency review
```

Migration Priority Order

```
  Tier 1 (migrate first): Simple pipelines, no shared libraries, standard
  tools
  Tier 2 (migrate next):  Shared library consumers (once lib is ported to
  Actions)
  Tier 3 (plan carefully): Complex dynamic pipelines, unique plugin
  dependencies
  Tier 4 (evaluate ROI):  Specialized toolchains, Jenkins-only integrations
  Tier 5 (stay on Jenkins): Air-gapped, mainframe, or irremovable Jenkins
  plugins
```

End of Category 10: Migration & Strategic Decisions

Complete Course Summary — All 10 Categories

```
Cat 1: Architecture, YAML, Events, Contexts      (fundamentals)
Cat 2: Actions — JS, Docker, Composite           (building blocks)
Cat 3: Reusability — Composite vs Reusable WF    (at scale)
Cat 4: Secrets, Environments, OIDC, Security     (trust + auth)
Cat 5: Matrix, Concurrency, Cache, Artifacts     (advanced patterns)
Cat 6: Self-Hosted Runners, ARC, Cost            (compute layer)
Cat 7: Docker, K8s, Terraform, Cloud, GitOps     (real workloads)
Cat 8: Monorepo — Path Filters, Detection, Tools (scale patterns)
Cat 9: Logs, Debug, Badges, API, Audit           (operations)
Cat 10: Migration, Comparison, Governance        (strategy)
```

Highest-yield interview topics across all categories:

1. OIDC keyless auth (Cat 4) — asked in virtually every senior interview
2. ARC architecture (Cat 6) — asked in platform/infra roles
3. Dynamic matrix from changed paths (Cat 5+8) — asked in scale discussions
4. Reusable Workflow vs Composite Action (Cat 3) — asked to test depth
5. pull_request_target security risk (Cat 4) — asked in security-conscious orgs
6. Jenkins migration mapping (Cat 10) — asked given your background
7. Concurrency cancel-in-progress semantics (Cat 5) — asked in deploy discussions
8. Ephemeral vs persistent runner security (Cat 6) — asked in security reviews